Leksion
Trashëgimia dhe Polimorfizmi në Python

Alion Emini

kurseinformatike.com

Përmbajtja

1. Trashëgimia
2. Polimorfizmi
3. `super()`
4. Atributet dhe metodat
5. "is-a" dhe "has-a"
6. Duck typing dhe `isinstance`
7. Përmbledhje

1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Trashëgimia: ideja bazë

Përkufizim

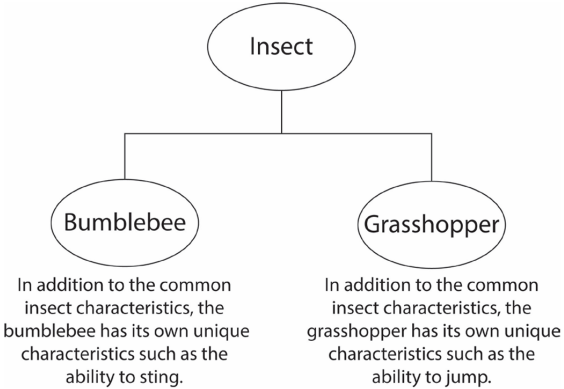
Trashëgimia (*inheritance*) lejon që një klasë e re të **zgjerojë** një klasë ekzistuese. Klasa e re **trashëgon** atributet dhe metodat e klasës që zgjeron.

- Klasa e përgjithshme quhet **superklasë**.
- Klasa më e specializuar quhet **nënklasë**.
- Nënklasa merr falas gjithë funksionalitetin e superklasës, dhe mund të shtojë gjëra të reja.

Përgjithësimi dhe specializimi

- Në botën reale kemi shpesh objekte **të përgjithshme** dhe versione të tyre **të specializuara**.
- Shembull: **insekti** si koncept i përgjithshëm.
  - Karkalecat dhe bletët *bumblebee* janë insekte.
  - Ata i **kanë** të gjitha karakteristikat e përgjithshme të një insekti.
  - **Plus** disa veçori të tyret (kërcejnë, thumb, ...).
- Në OOP:
  - `Insect` ⇒ klasë e përgjithshme;
  - `Grasshopper`, `Bumblebee` ⇒ nënklasa të specializuara.

Insekti dhe nënllojtë e tij



Bletët dhe karkalecat janë versione të specializuara të një insekti.

Marrëdhënia "është një" (*is-a*)

Përkufizim

Kur një objekt është version i **specializuar** i një tjetri, themi që ka marrëdhënie **është një** (*is-a*).

- Shembuj:
  - karkaleci **është një** insekt;
  - *poodle* **është një** qen;
  - makinë **është një** automjet;
  - drejtkëndësh **është një** formë;
  - futbollist **është një** sportist.
- Objekti i specializuar:
  - ka **të gjitha** karakteristikat e atij të përgjithshëm;
  - **plus** disa karakteristika shtesë.

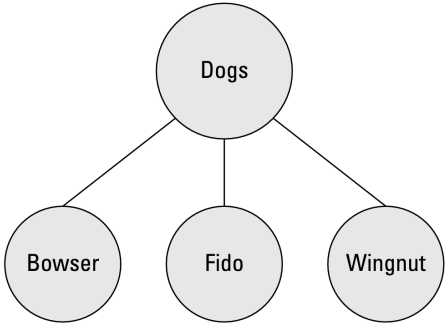
Superklasa dhe nënklasa

- **Superklasa** (*superclass* / *base class*):
  - përshkruan sjelljen dhe të dhënat **e përgjithshme**;
- **Nënklasa** (*subclass* / *derived class*):
  - është version i **specializuar** i superklasës;
  - trashëgon atributet dhe metodat;
  - mund të shtojë atributet/metoda të reja ose të rishkruajë (*override*) metoda ekzistuese.

Shembull në gjuhë natyrore

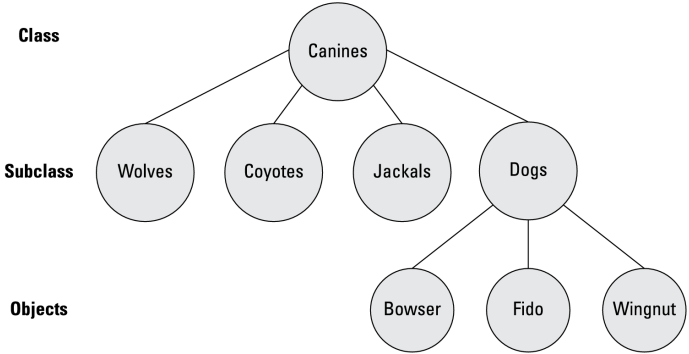
`Car` **është një** `Automobile`.
`Dog` **është një** `Mammal`.

Qentë si objekte të klasës Dog



Qentë janë instanca të një klase më të përgjithshme.

Nga Dog te Canine, Mammal, ...



- Wolves, Coyotes, Jacklas, Dogs ⇒ Canine.
- Canine është nënklasë e Mammal.
- Mammal është nënklasë e Animal.
- Animal është nënklasë e LivingThing.

Hierarkia nga e përgjithshmja tek e veçanta

- Shembulli me qen: `LivingThing` → `Animal` → `Mammal` → `Canine` → `Dog`
- E njëjta ide vlen edhe për:
  - mace (*felines*: `Cat`, `Lion`, `Tiger`, ...);
  - mjetet e transportit: `Vehicle` → `Automobile` → `Car`, `Truck`, `SUV`.
- Trashëgimia në Python sjell këtë strukturë *hierarkie* në kod.

Trashëgimia nga këndvështrimi i kodit

- Superklasa përkufizon:
  - **atributet e përbashkëta** (p.sh. `make`, `model`, `price`);
  - **metodat e përbashkëta** (p.sh. `get_price()`, `set_model()`).
- Nënklasa përkufizon:
  - vetëm gjërat që kanë kuptim **vetëm për atë lloj**;
  - p.sh. `doors` për `Car`, `drive_type` për `Truck`.
- Nënklasa mund:
  - të përdorë metodat e superklasës ashtu si janë;
  - të shtojë metoda të reja;
  - të **rishkruajë** (*override*) metoda ekzistuese.

Skenari: autosalloni i makinave

Skenari i shembullit

Po ndërtojmë një program për një autosallon (*car dealership*) që menaxhon inventarin.

- Llojet e automjeteve:
  - makina të zakonshme (`Car`);
  - pickup trucks (`Truck`);
  - `SUV`.
- Për çdo automjet ruhen të dhënat:
  - `make`, `year model`, `mileage`, `price`.
- Çdo lloj ka edhe të dhëna *të veçanta*:
  - `Car`: numri i dyerve;
  - `Truck`: tipi i lëvizjes (`'2WD'`, `'4WD'`);
  - `SUV`: kapaciteti i pasagjerëve.

Qasja naive (pa trashëgimi)

- Mund të krijonim 3 klasa të pavarura:
  - `Car` me `make`, `model`, `mileage`, `price`, `doors`;
  - `Truck` me `make`, `model`, `mileage`, `price`, `drive_type`;
  - `SUV` me `make`, `model`, `mileage`, `price`, `pass_cap`.
- **Problemi:**
  - shumë kod i kopjuar;
  - çdo ndryshim te të dhënat e përbashkëta kërkon modifikimin e 3 klasave.

Qasja me trashëgimi

- Krijojmë një superklasë `Automobile`:
  - mban `make`, `model`, `mileage`, `price`;
  - përkufizon `get\_` / `set\_` për këto.
- Pastaj krijojmë nënklasa:
  - `Car(Automobile)` shton `doors`;
  - `Truck(Automobile)` shton `drive_type`;
  - `SUV(Automobile)` shton `pass_cap`.
- E përbashkëta shkruhet vetëm një herë në `Automobile`.

Programi 1 - Klasa Automobile (vehicles.py) I

```
class Automobile:
    def __init__(self, make, model, mileage, price):
        self.__make = make
        self.__model = model
        self.__mileage = mileage
        self.__price = price

    def set_make(self, make):
        self.__make = make

    def set_model(self, model):
        self.__model = model

    def set_mileage(self, mileage):
        self.__mileage = mileage

    def set_price(self, price):
        self.__price = price
```

Programi 1 - Klasa Automobile (vehicles.py) II

```
    def get_make(self):
        return self.__make

    def get_model(self):
        return self.__model

    def get_mileage(self):
        return self.__mileage

    def get_price(self):
        return self.__price
```

Analiza e klasës `Automobile`

- Atribute private:
  - `__make`, `__model`, `__mileage`, `__price`.
- Metoda `__init__`:
  - merr të dhënat e përgjithshme;
  - inicializon atributet private.
- `set_...` janë **mutators** (ndryshojnë gjendjen).
- `get_...` janë **accessors** (kthejnë vlera, nuk ndryshojnë gjendjen).
- `Automobile` është klasë e plotë: mund të krijojmë objekte direkt prej saj.

Programi 2 - Klasa Car (vehicles.py)

```
class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        Automobile.__init__(self, make, model, mileage, price)
        self.__doors = doors

    def set_doors(self, doors):
        self.__doors = doors

    def get_doors(self):
        return self.__doors
```

Analiza e klasës Car

- Header-i:
  - `class Car(Automobile):`
    ⇒ `Car` është nënklasë e `Automobile`.
- Marrëdhënia: `Car` **është një** `Automobile`
- `__init__` i nënklasës:
  - thërret `Automobile.__init__` për pjesën e përgjithshme;
  - krijon atributin e ri `__doors`.
- Metodat `get_make`, `get_model`, ... nuk përcaktohen këtu: **trashëgohen** nga `Automobile`.

Programi 3 - car_demo.py I

```python
import vehicles

def main():
    used_car = vehicles.Car('Audi', 2007, 12500, 21500.0, 4)

    print('Make:', used_car.get_make())
    print('Model:', used_car.get_model())
    print('Mileage:', used_car.get_mileage())
    print('Price:', used_car.get_price())
    print('Number of doors:', used_car.get_doors())

if __name__ == '__main__':
    main()
```

Programi 3 - car_demo.py II

```
Make: Audi
Model: 2007
Mileage: 12500
Price: 21500.0
Number of doors: 4
```

Programi 4 - Klasa Truck (vehicles.py)

```
class Truck(Automobile):
    def __init__(self, make, model, mileage, price, drive_type):
        Automobile.__init__(self, make, model, mileage, price)
        self.__drive_type = drive_type

    def set_drive_type(self, drive_type):
        self.__drive_type = drive_type

    def get_drive_type(self):
        return self.__drive_type
```

Programi 5 - Klasa SUV (vehicles.py)

```
class SUV(Automobile):
    def __init__(self, make, model, mileage, price, pass_cap):
        Automobile.__init__(self, make, model, mileage, price)
        self.__pass_cap = pass_cap

    def set_pass_cap(self, pass_cap):
        self.__pass_cap = pass_cap

    def get_pass_cap(self):
        return self.__pass_cap
```

Programi 6 - car_truck_suv_demo.py I

```python
import vehicles

def main():
    car = vehicles.Car('BMW', 2001, 70000, 15000.0, 4)
    truck = vehicles.Truck('Toyota', 2002, 40000, 12000.0, '4WD')
    suv = vehicles.SUV('Volvo', 2000, 30000, 18500.0, 5)

    print('The following car is in inventory:')
    print('Make:', car.get_make())
    print('Model:', car.get_model())
    print('Mileage:', car.get_mileage())
    print('Price:', car.get_price())
    print('Number of doors:', car.get_doors())
    print()

    print('The following pickup truck is in inventory.')
    print('Make:', truck.get_make())
```

Programi 6 - car_truck_suv_demo.py II

```
    print('Model:', truck.get_model())
    print('Mileage:', truck.get_mileage())
    print('Price:', truck.get_price())
    print('Drive type:', truck.get_drive_type())
    print()

    print('The following SUV is in inventory.')
    print('Make:', suv.get_make())
    print('Model:', suv.get_model())
    print('Mileage:', suv.get_mileage())
    print('Price:', suv.get_price())
    print('Passenger Capacity:', suv.get_pass_cap())

if __name__ == '__main__':
    main()
```

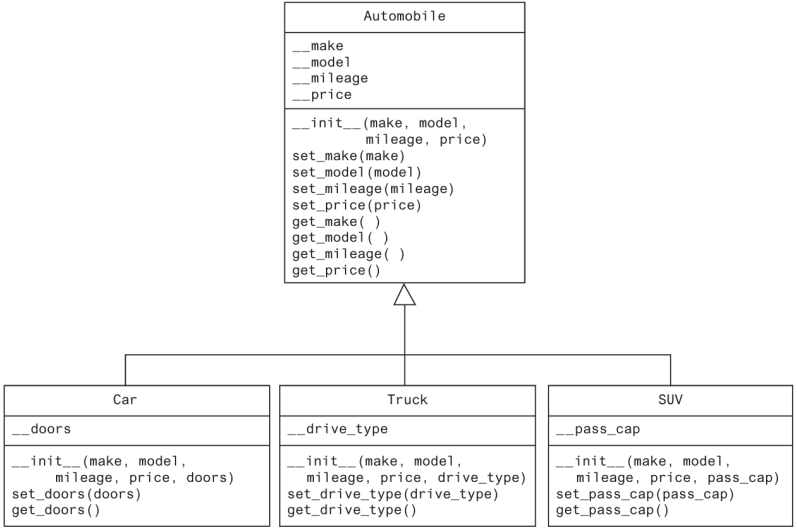
Shembull output-i (car_truck_suv_demo.py)

```
The following car is in inventory:
Make: BMW
Model: 2001
Mileage: 70000
Price: 15000.0
Number of doors: 4

The following pickup truck is in inventory.
Make: Toyota
Model: 2002
Mileage: 40000
Price: 12000.0
Drive type: 4WD

The following SUV is in inventory.
Make: Volvo
Model: 2000
Mileage: 30000
Price: 18500.0
Passenger Capacity: 5
```

Trashëgimia në UML

- Në UML, çdo klasë përfaqësohet si drejtkëndësh (emër, atribute, metoda).
- Trashëgimia paraqitet me:
  - një vijë që lidh nënklasën me superklasën;
  - një shigjetë trekëndore bosh që tregon nga **superklasa**.
- Lexohet: `SubClass` **është një** `SuperClass`.

UML: Automobile, Car, Truck, SUV



1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Polimorfizmi: koncept

Përkufizim

Polimorfizmi (*polymorphism*) lejon që nënklasat të kenë **metoda me të njëjtin emër** si superklasa, por me **sjellje të ndryshme**. Programi zgjedh **versionin e duhur** sipas **llojit të objektit**.

- Një **interface i njëjtë** (p.sh. emri i metodës) ⇒ zbatime të ndryshme në klasa të ndryshme.

Dy përbërësit kryesorë të polimorfizmit

1. **Rishkrimi i metodave** (*method overriding*)
   - metoda përkufizohet në superklasë;
   - nënklasa përkufizon një metodë me të njëjtin emër, që mbulon versionin e superklasës.
2. **Thirrja e versionit të duhur**
   - kur thërret metodën përmes një objekti, Python zgjedh automatikisht versionin sipas llojit të objektit;
   - **tipi real** i objektit vendos se cili kod ekzekutohet.

Programi 7 - animals.py (Mammal, Dog, Cat) I

```
class Mammal:
    def __init__(self, species):
        self.__species = species

    def show_species(self):
        print('I am a', self.__species)

    def make_sound(self):
        print('Grrrrr')
```

Programi 7 - animals.py (Mammal, Dog, Cat) II

```
class Dog(Mammal):
    def __init__(self):
        Mammal.__init__(self, 'Dog')

    def make_sound(self):
        print('Woof! Woof!')
```

Programi 7 - animals.py (Mammal, Dog, Cat) III

```
class Cat(Mammal):
    def __init__(self):
        Mammal.__init__(self, 'Cat')

    def make_sound(self):
        print('Meow')
```

Çfarë po ndodh në `animals.py`?

- `Mammal`:
  - ruan `__species`;
  - `show_species()` shfaq I am a ...;
  - `make_sound()` jep një tingull të përgjithshëm.
- `Dog(Mammal)` dhe `Cat(Mammal)`:
  - trashëgojnë `show_species()`;
  - **rishkruajnë** `make_sound()` me tingujt tipikë.
- Polimorfizmi:
  - emri i njëjtë i metodës: `make_sound()`;
  - sjellje të ndryshme: qen, mace, gjitar i përgjithshëm.

Shembull përdorimi i animals.py I

```
import animals

mammal = animals.Mammal('regular mammal')
mammal.show_species()
mammal.make_sound()

dog = animals.Dog()
dog.show_species()
dog.make_sound()

cat = animals.Cat()
cat.show_species()
cat.make_sound()
```

Shembull përdorimi i animals.py II

```
I am a regular mammal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
```

Programi 9 - polymorphism_demo.py

```python
import animals

def main():
    mammal = animals.Mammal('regular animal')
    dog = animals.Dog()
    cat = animals.Cat()

    show_mammal_info(mammal)
    show_mammal_info(dog)
    show_mammal_info(cat)

def show_mammal_info(creature):
    creature.show_species()
    creature.make_sound()

if __name__ == '__main__':
    main()
```

Shembull output-i (polymorphism_demo.py)

```
I am a regular animal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
```

- `show_mammal_info` nuk kontrollon llojin.
- Thërret thjesht `show_species()` dhe `make_sound()`.
- Python zgjedh automatikisht versionin e duhur të `make_sound`.

Programi 10 - wrong_type.py

```
def main():
    show_mammal_info('I am a string')

def show_mammal_info(creature):
    creature.show_species()
    creature.make_sound()

if __name__ == '__main__':
    main()
```

Çfarë ndodh?

Objektet `str` *nuk* kanë `show_species()` dhe `make_sound()`. Gjatë ekzekutimit, Python hedh një `AttributeError`.

Programi 11 - polymorphism_demo2.py I

```
import animals

def show_mammal_info(creature):
    if isinstance(creature, animals.Mammal):
        creature.show_species()
        creature.make_sound()
    else:
        print('That is not a Mammal!')
```

Programi 11 - polymorphism_demo2.py II

```
def main():
    mammal = animals.Mammal('regular animal')
    dog = animals.Dog()
    cat = animals.Cat()

    show_mammal_info(mammal)
    show_mammal_info(dog)
    show_mammal_info(cat)

    show_mammal_info('I am a string')

if __name__ == '__main__':
    main()
```

Programi 11 - polymorphism_demo2.py III

```
I am a regular animal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
That is not a Mammal!
```

1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Thirrja e konstruktorit të superklasës (forma klasike)

```
class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        Automobile.__init__(self, make, model, mileage, price)
        self.__doors = doors
```

- Përdorim emrin e superklasës `Automobile` për të thirrur `__init__`.
- Funksionon, por repeton emrin e klasës kudo.

Programi 12 - vehicles2.py - super() - mënyra e rekomanduar

```
1 class Car(Automobile):
2     def __init__(self, make, model, mileage, price, doors):
3         super().__init__(make, model, mileage, price)
4         self.__doors = doors
```

Pse super()?

- Kodi bëhet më i pastër dhe më i lexueshëm.
- Nuk përsërisim emrin e superklasës.
- Nëse emri i superklasës ndryshon, super() vazhdon të funksionojë.
- Është i domosdoshëm në hierarki më komplekse trashëgimie.

Zgjerimi i metodës së superklasës (Programi 13 - vehicles3.py)

```
class Automobile:
    def __init__(self, make, model, mileage, price):
        self.__make = make
        self.__model = model
        self.__mileage = mileage
        self.__price = price

    def show_info(self):
        print('Make:', self.__make)
        print('Model:', self.__model)
        print('Mileage:', self.__mileage)
        print('Price:', self.__price)
```

Car.show_info: thirr prindin & shto detaje

```
class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        super().__init__(make, model, mileage, price)
        self.__doors = doors

    def show_info(self):
        super().show_info()
        print('Number of doors:', self.__doors)
```

- `Car.show_info` **nuk** kopjon kodin e `Automobile.show_info`.
- Fillimisht thërret `super().show_info()`.
- Pastaj shton rreshtin me numrin e dyerve.

1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Atribute instance vs atribute klase

Atributet e instancës

- Krijohen zakonisht në `__init__` me `self.x`.
- Çdo objekt ka **kopjen e vet** të vlerës.

Atributet e klasës

- Përkufizohen brenda klasës, por jashtë metodave.
- Janë të **përbashkëta** për të gjitha instancat.

Programi 14 - dogs.py I

```
class Dog:
    # Atribut i klasës: i përbashkët për të gjithë qentë
    species = 'Canis familiaris'

    def __init__(self, name, age):
        # Atribute të instancës
        self.name = name
        self.age = age
```

Programi 14 - dogs.py II

```
def main():
    d1 = Dog('Luna', 3)
    d2 = Dog('Rex', 5)

    print(d1.name, d1.age, d1.species)
    print(d2.name, d2.age, d2.species)

    print('Specia e të gjithë qenve është:', Dog.species)

if __name__ == '__main__':
    main()
```

Output dhe interpretim

```
Luna 3 Canis familiaris
Rex 5 Canis familiaris
Specia e të gjithë qenve është: Canis familiaris
```

- name, age ndryshojnë nga një qen te tjetri ⇒ atribute instance.
- species është e njëjtë për të gjithë ⇒ atribut klase.
- Lexohet si Dog.species, por mund të aksesohet edhe si d1.species.

Programi 14 - dogs2.py

```
class Automobile:
    # Atribut i klasës: numri total i automjeteve
    count = 0

    def __init__(self, make, model):
        self.make = make
        self.model = model
        Automobile.count += 1

def main():
    a1 = Automobile('Audi', 2007)
    a2 = Automobile('BMW', 2015)
    a3 = Automobile('Toyota', 2020)

    print('Janë krijuar', Automobile.count, 'automjete.')

if __name__ == '__main__':
    main()
```

Programi 15 - dogs3.py - Atributet private të klasave I

```
class Automobile:
    # Atribut i klasës: numri total i automjeteve të krijuara
    __count = 0

    @classmethod
    def count(cls):
        return cls.__count

    def __init__(self, make, model):
        self.__make = make
        self.__model = model

        # Sa herë krijohet një objekt i ri, rrisim numërimin
        Automobile.__count += 1
```

Programi 15 - dogs3.py - Atributet private të klasave II

```
def main():
    a1 = Automobile('Audi', 2007)
    a2 = Automobile('BMW', 2015)

    print('Janë krijuar', Automobile.count(), 'automjete.')
if __name__ == '__main__':
    main()
```

Dy llojet kryesore të metodave

Metodat e instancës

- Parametri i parë: `self`.
- Punojnë me një **objekt konkret**.

Metodat e klasës (`@classmethod`)

- Parametri i parë: `cls`.
- Shënohen me dekoratorin `@classmethod`.
- Punojnë me **klasën** (dhe atributet e saj të klasës).

Shembull: metoda instance te Car (Programi 16 - automobile.py)

```
class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        super().__init__(make, model, mileage, price)
        self.__doors = doors

    def get_doors(self):
        return self.__doors

    def show_info(self):
        print('Make:', self.get_make())
        print('Model:', self.get_model())
        print('Doors:', self.__doors)
```

- Këto janë metoda që punojnë me *një* makinë konkrete (car1, car2, ...).

Programi 16 - automobile.py - Metodë klase: how_many

```
class Automobile:
    count = 0    # atribut klase

    def __init__(self, make, model):
        self.make = make
        self.model = model
        Automobile.count += 1

    @classmethod
    def how_many(cls):
        print('Deri tani janë krijuar', cls.count, 'automjete.')
```

Programi 16 - automobile.py - Thirrja e metodës së klasës

```
def main():
    Automobile.how_many()    # thirrje përmes klasës

    a1 = Automobile('Audi', 2007)
    a2 = Automobile('BMW', 2015)

    Automobile.how_many()    # tani count është 2
    a2.how_many()            # funksionon

if __name__ == '__main__':
    main()
```

- Edhe kur thirret përmes një instance, parametri i parë është `cls` (klasa), jo `self`.
- E përdorim për logjikë që vlen për **llojin** e objekteve, jo për një objekt të vetëm.

Programi 17 - automobile2.py - Metodë klase si konstruktor alternativ

```python
class Automobile:
    def __init__(self, make, model):
        self.__make = make
        self.__model = model

    @classmethod
    def from_string(cls, text):
        __make, __model_str = text.split('-')
        model = int(__model_str)
        return cls(__make, model)

    def __str__(self):
        return f"{self.__make} {self.__model}"

def main():
    auto1 = Automobile.from_string('Audi-2007')
    print(auto1)

if __name__ == '__main__':
    main()
```

1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Trashëgimi vs kompozitë

Marrëdhënia "is-a" (*është një*)

- Përdoret kur një klasë është një **lloj më i specializuar** i një tjetre.
- Implementohet me **trashëgimi**: `class Car(Automobile): ...`

Marrëdhënia "has-a" (*ka një*)

- Përdoret kur një objekt **përbëhet** nga objekte të tjera.
- Implementohet me **kompozitë**: një objekt ruan të tjerë si atribute (`self.engine`, `self.address`, ...).

Programi 18 - vehicles4.py I

```
class Engine:
    def __init__(self, horsepower):
        self.__horsepower = horsepower

    def start(self):
        print('Engine starting...')

    def get_horsepower(self):
        return self.__horsepower
```

Programi 18 - vehicles4.py II

```
class Automobile:
    def __init__(self, make, model):
        self.__make = make
        self.__model = model

    def show_basic_info(self):
        print('Make:', self.__make)
        print('Model:', self.__model)
```

Programi 18 - vehicles4.py III

```
class Car(Automobile):    # Car IS-A Automobile
    def __init__(self, make, model, engine):
        super().__init__(make, model)
        # Car HAS-A Engine
        self.__engine = engine

    def start(self):
        print('Starting car...')
        self.__engine.start()

    def show_info(self):
        self.show_basic_info()
        print('Horsepower:', self.__engine.get_horsepower())
```

Programi 18 - vehicles4.py IV

```
def main():
    engine = Engine(150)
    my_car = Car('Toyota', 2020, engine)

    my_car.show_info()
    my_car.start()

if __name__ == '__main__':
    main()
```

Kur të përdorim cilën?

- Përdor **trashëgiminë** kur:
  - fjalia `SubClass` *është një* `SuperClass` ka kuptim;
  - p.sh. `Car` është një `Automobile`.
- Përdor **kompozitën** kur:
  - fjalia `ClassA` *ka një* `ClassB` përshkruan më mirë marrëdhënien;
  - p.sh. `Car` ka një `Engine`.
- Rregull praktik:
  - Nëse nuk mund ta thuash mirë me *është një*, mos përdor trashëgimi, përdor kompozitë.

1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Parimi i *duck typing*

Shprehja klasike

Nëse ecën si rosë dhe bën zhurmë si rosë, atëherë është rosë.

- Në Python:
  - nuk na intereson shumë *lloji* formal i objektit;
  - rëndësi ka që objekti të ketë **metodat** që presim;
  - nëse i ka, e përdorim; nëse jo, marrim gabim.

Programi 19 - duck.py I

```
class Dog:
    def make_sound(self):
        print('Woof! Woof!')

class Cat:
    def make_sound(self):
        print('Meow')

class Robot:
    def make_sound(self):
        print('Beep boop!')
```

Programi 19 - duck.py II

```python
def play_sound(creature):
    # Duck typing: supozojmë që ka make_sound()
    creature.make_sound()


def main():
    d = Dog()
    c = Cat()
    r = Robot()

    play_sound(d)  # Woof! Woof!
    play_sound(c)  # Meow
    play_sound(r)  # Beep boop!

if __name__ == '__main__':
    main()
```

Çfarë po ndodh këtu?

- `Dog`, `Cat`, `Robot` **nuk** trashëgojnë nga një superklasë e përbashkët.
- `play_sound` nuk përdor `isinstance`.
- I vetmi kërkim:
  - objekti të ketë metodën `make_sound()`.
- Ky është **duck typing**: për sa kohë sillet si rosë, e trajtojmë si të tillë.
- Nëse i kalojmë funksionit `play_sound` një objekt që *nuk* ka metodën `make_sound()`, do të marrim `AttributeError` në kohë ekzekutimi.

isinstance: kontroll i qartë i tipit

- Funksioni i integruar `isinstance(obj, Class)` kthen `True` nëse `obj` është:
  - instancë e `Class`, **ose**
  - instancë e një **nënklase** të `Class`.
- Përdoret kur:
  - duam të lejojmë *vetëm* objekte të një hierarkie të caktuar;
  - duam të shmangim gabimet në kohë ekzekutimi (`AttributeError`) me një mesazh më të qartë.
- Shembull tipik: kontrollojmë nëse diçka është `Mammal` përpara se ta trajtojmë si të tillë.

Shembull: isinstance në veprim

```python
import animals

def play_mammal_sound(creature):
    if isinstance(creature, animals.Mammal):
        creature.show_species()
        creature.make_sound()
    else:
        print('Ky objekt nuk është gjitar (Mammal).')

def main():
    dog = animals.Dog()
    cat = animals.Cat()
    text = 'Jam një string'

    play_mammal_sound(dog)
    play_mammal_sound(cat)
    play_mammal_sound(text)

if __name__ == '__main__':
    main()
```

Output-i i shembullit me isinstance

```
I am a Dog
Woof! Woof!
I am a Cat
Meow
Ky objekt nuk është gjitar (Mammal).
```

- Për Dog dhe Cat, kushti isinstance(creature, animals.Mammal) është True.
- Për text (string), kushti është False dhe shfaqet mesazhi ynë i personalizuar.
- Kështu kontrollojmë llojin **përpara** se të thërrasim metoda.

Duck typing vs isinstance: kur cilën?

Duck typing

- Nuk kontrollojmë llojin, vetëm *sjelljen*.
- Më fleksibël, më pythonic.
- E përshtatshme kur:
  - kemi kontroll mbi të gjithë kodin;
  - është në rregull të marrim `AttributeError` nëse mungon metoda.

isinstance

- Kontrollojmë qartë që objekti i përket një hierarkie të caktuar.
- E përshtatshme kur:
  - ndërveprojmë me kod të jashtëm / input nga përdoruesi;
  - duam mesazhe gabimi më miqësore dhe më të kontrolluara.

Rregull praktik

- Fillimisht mendo me **duck typing**:
  - A ka objekti metodat që më duhen?
- Nëse rreziku i input-eve të çuditshme është i madh, shto një **kufi sigurie** me `isinstance`.
- Mos e mbingarko kodin me kontrolle tipash nëse nuk është e nevojshme:
  - Python nuk është Java/C#;
  - fleksibiliteti është një nga avantazhet kryesore.

1 Trashëgimia

2 Polimorfizmi

3 super()

4 Atributet dhe metodat

5 "is-a" dhe "has-a"

6 Duck typing dhe isinstance

7 Përmbledhje

Çfarë mësuam në këtë leksion

- **Trashëgimia**:
  - superklasa vs nënklasa;
  - marrëdhënia *është një* (`is-a`).
- **Polimorfizmi**:
  - rishkrimi i metodave (*overriding*);
  - zgjedhja automatike e versionit sipas llojit të objektit.
- `super()`:
  - thirrja e konstruktorit dhe metodave të superklasës pa përsëritur emrin e saj.
- **Atributet / metodat e klasës**:
  - të përbashkëta për të gjitha instancat (`count`, `@classmethod`).
- **Marrëdhëniet `is-a` dhe `has-a`**:
  - trashëgimi vs kompozitë.
- **Duck typing dhe `isinstance`**:
  - fleksibilitet vs kontrolle eksplisite të tipit.

Çfarë mund të provosh vetë

- Krijo një hierarki klasash p.sh. `Shape`, `Rectangle`, `Circle`:
  - përkufizo një metodë `area()` në superklasë;
  - rishkruaje në nënklasa.
- Krijo një funksion `print_area(shape)` që thërret `area()` - shiko polimorfizmin në veprim.
- Shto një kontroll me `isinstance` që pranon vetëm forma që janë `Shape`.
- Eksperimento me atribute klase për të numëruar sa forma janë krijuar.

Pyetje?

Faleminderit për vëmendjen!

Pyetje, paqartësi, komente?